

LoopLight — Week 3 Project Documentation

Mastering Agentic AI Certification | Agentic AI Systems | Track: No-code (n8n)

1. Project Overview

LoopLight is a personal attention-management agent I built to solve a problem I actually have: too many half-formed thoughts, tasks, and worries floating around, and no easy way to decide what to actually do next. You dump whatever's on your mind into a form — messy, unstructured, however it comes out — and the system sorts it into real, persistent items. When you're ready for a recommendation, it looks at everything still open, weighs it against how much time and energy you actually have right now, and hands you up to three things worth doing — not a dump of your whole list back at you. The goal is not triaging or capturing the brain dump — but less figuring out what to do and more time spent actually doing it.

The one thing I cared about most: it never acts without me. It can suggest, but a task only ever moves forward because I picked it.

2. The One-Liner

My agent helps me triage a running brain-dump into a short, ranked set of next-action recommendations inside an n8n form, replacing the mental overhead of manually re-reading and re-prioritizing scattered notes myself. It does the recommending on its own using tool calls against my own persistent task memory, hands off to me before anything is marked in progress, and I'll know it works when I can go from a raw brain dump to a chosen next action in under 2 minutes, with a recommendation I'd actually act on most of the time.

3. The Agent Framework

Field	Answer
Agent goal	Takes a free-text brain dump, then a lightweight capacity check-in, and recommends up to 3 open items worth doing right now.
Where used	Two connected n8n Forms: an intake form (“What's on your mind?”) and a follow-up capacity/recommendation form.
Steps, in order	1) Capture the brain dump. 2) Extract structured items (LLM). 3) Log the session. 4) Save any new items as ‘open’ rows, regardless of capacity. 5) Ask a quick, casual capacity question (“Got it!”). 6) If capacity is given, the Decision Agent reads all open items via a tool and ranks up to 3 by urgency, deadline, and capacity fit. 7) Recommendations shown on a review form. 8) On my pick, that item’s status is updated to ‘In Progress.’
What it can do (tools)	Read Open Items (Google Sheets, read-only — the only tool the Decision Agent currently has). Write Session / Save Items / Update Item Status (Google Sheets, write — all fixed-logic writes except the final status update, which is gated by my approval on the review form).
What it remembers	Every extracted item persists across sessions in a Google Sheet (Items tab), tagged open or In Progress. Capacity check-ins persist separately in a Sessions tab. This is real cross-session memory, not just single-conversation context.
What it should never do	Never mark an item In Progress without my explicit selection on the review form. Never invent a deadline or priority that wasn’t in the source text. Never silently drop a malformed submission or lose brain-dump items just because capacity wasn’t given.

Human-in-the-loop	After the Decision Agent proposes up to 3 candidates, I review them on a form and either pick one (→ In Progress) or choose ‘None of the above’ (→ no change). That's the only state-changing write in the whole system, and it only happens on my say-so.
What happens when something breaks	A missing or malformed form submission returns a clear message instead of crashing the run. A capacity-only check-in (no items) is treated as a valid no-op, not an error. If the Decision Agent's output doesn't parse into valid JSON, a fallback ‘nothing fits right now’ message is shown instead of an unhandled error.
How I know it worked	I can go from a raw brain dump to picking one clear next action in under 2 minutes, and the recommendation is one I'd actually act on — not a generic restatement of my whole list.

4. Architecture Pattern

This is a single ReAct agent embedded inside a deterministic pipeline, not multi-agent orchestration. Only one node — the Decision Agent — exercises real judgment: it has to call a tool (Read Open Items) and weigh competing factors (urgency, my stated capacity, deadlines) to decide what to surface. Everything else — extraction, sheet writes, branching on form input — is fixed logic, even though some of it runs through an LLM (a one-shot extraction call isn't the same thing as an agent making a decision).

I chose this on purpose, not as a shortcut. There's exactly one judgment call in this task — ‘which item(s) fit right now’ — not multiple specialized roles that need to hand work to each other. A multi-agent setup would have added coordination overhead without adding real capability. The handout explicitly lists single ReAct as a valid pattern alongside multi-agent, pipeline, and voice, so this is a deliberate scope decision, not a gap.

5. Flow Diagram

Full end-to-end flow, left to right — see the landscape page immediately following this one for a larger, readable version.

6. Datasets Used

- No external or public dataset — the only data is my own brain-dump submissions, captured live through the form.
- Persistent state lives in two Google Sheets tabs: Sessions (capacity check-ins over time) and Items (every extracted item, tagged open or In Progress).

7. Prompts Used (Vibe Coding)

Information Extractor — purpose

Given a raw brain-dump submission, pull out zero or more structured items — each with a description, item type, deadline text (if any), effort estimate, and a needs-clarification flag — without inventing details that aren't actually there. It's also told explicitly not to manufacture a task out of filler text (“nothing,” “just checking in”) — that should return zero items, not a fake one.

Decision Agent — core policy (paraphrased from the live system prompt)

- Always call Read Open Items before recommending anything — never recommend from memory alone.
- Identify tasks with explicit urgency or deadlines first, and never hide an urgent task just because current capacity is low.
- If multiple urgent tasks exist, surface up to three and let me choose — don't guess which one matters most.
- If nothing is urgent, recommend something that actually fits my current time and energy.
- Only recommend rest as the sole answer when nothing urgent exists and nothing reasonably fits.
- Never create, update, complete, or delete anything directly — only ever propose.

- Always return exactly 3 ranked candidates when at least 3 open items exist, being honest in the ‘why’ field when one is a weaker fit rather than hiding that it's a stretch.
- Return the recommendation as strict JSON, enforced through n8n's Structured Output Parser rather than relying on the model to format it correctly unprompted.

8. Iterations and Design Decisions

I want to be upfront that this build went through a lot more churn than the final architecture suggests — documenting the ‘why not’ alongside the ‘what shipped’ felt more honest than pretending it arrived this way in one pass.

8.1 Todoist as the item store (considered, then reversed)

Early on I explored moving items out of Sheets and into Todoist, using labels for effort and item type, with a second ‘Classification Agent’ to check new items against existing Todoist tasks. I scoped this out — it needed a second agentic component, a new tool, and a new label system, which was too much new surface area for the timeline. Sheets stayed the single source of truth.

8.2 Re-ask loop vs. ranked top-3

My first design for the approval step was a loop: show one recommendation, let me say ‘no, give me another,’ re-run the agent excluding what I'd rejected, capped at 3 tries. That needed real cyclical state (a rejected-items list, an attempt counter) with real failure modes — stuck loops, lost state between passes — for what amounted to a cosmetic difference in experience. I replaced it with a single Decision Agent call that returns up to 3 ranked candidates at once. Same result from my side, none of the cyclical-state risk.

8.3 Status label: ‘completed’ vs. ‘accepted’ vs. ‘In Progress’

Picking a recommendation doesn't mean the task is actually done, so labelling it ‘completed’ on approval would be a false claim. I initially settled on ‘accepted’ as an honest, terminal label — it just means I took ownership, not that it's finished. In the final build I renamed the value to ‘In Progress,’ which reads more naturally in the sheet and means the same thing functionally: the item drops out of future recommendations the moment I pick it, without claiming it's done. Known limitation, unchanged either way: an item marked In Progress that never actually gets finished will not resurface, since the Decision Agent only reads status = open.

8.4 Error handling

The item-preparation step originally threw a hard error on any missing or malformed input, which crashed the whole run — including a perfectly valid capacity-only check-in with no items attached. I rewrote it so only a genuinely unreadable submission (missing timestamp) returns a graceful error message; a missing or empty items array is now treated as a valid no-item submission, not a failure.

8.5 Capacity flow redesign

Originally, capacity was collected on the same form as the brain dump, and a missing capacity value silently blocked the item-saving step too — meaning a capacity-less brain dump could lose the items entirely. I moved the capacity gate to sit after item-saving instead of before it, so items always persist regardless of capacity, and added a friendly, unconditional follow-up screen (‘Got it!’) that asks for capacity separately, with a casual multiple-choice format instead of free text, including an explicit ‘skip’ option rather than relying on an ambiguous blank submission.

9. Testing and Validation

The handout doesn't require a formal eval suite for the no-code track — that's called out specifically as a code-track option (LangSmith evals). What I did instead was manually exercise every branch of the flow during the build, listed here as evidence against the ‘recovers from errors’ and ‘decides what to do next’ requirements.

Scenario	Expected behavior	Result
----------	-------------------	--------

Brain dump with real items, capacity given	Items saved as open; Decision Agent ranks and recommends	Pass
Brain dump only, no capacity (skip on follow-up)	Items still saved as open; no recommendation triggered	Pass, after moving the capacity gate below the save step
Capacity-only check-in, no items	Session logged; no items created; no crash	Pass, after removing the hard throw in Prepare Items
Picking a real recommended item	That item's status updates to In Progress in Sheets	Pass
Picking 'None of the above'	No write occurs; workflow ends cleanly	Pass, after fixing an IF-node type mismatch (Boolean vs. String) and a missing Matching Column on the Update step
Fewer than 3 open items exist	Fewer than 3 candidates returned, no blank/garbled options	Pass, after tightening the prompt to only under-return when genuinely fewer than 3 exist
Malformed/empty form submission	Graceful fallback message; workflow doesn't crash	Logic in place (Prepare Items fallback); not deliberately forced with a broken payload
Ambiguous item forcing a judgment call between two tools	Agent could flag instead of guessing	Designed, not shipped — see Known Limitations

10. Known Limitations and Gaps

An honest list of what didn't make it into the final build, found while auditing the exported workflow against this documentation:

- The capacity value collected on the 'Got it!' follow-up screen currently gates whether the Decision Agent runs at all, but is not yet passed into the Decision Agent's own reasoning prompt — it still reads capacity from the original Information Extractor output, which, now that the intake form only asks for a brain dump, will typically come back as unspecified. The agent still reasons correctly on urgency and deadlines; it just isn't using my stated energy/time level as precisely as the design intends. A quick fix would be threading the follow-up capacity value into the Decision Agent's input alongside the extracted items.
- A second tool for the Decision Agent (a 'Flag for Clarification' action, letting it write back to Sheets instead of guessing at a vague item) was designed but not implemented in the shipped workflow — the agent currently has one tool, not two.
- Anchoring the Decision Agent's date reasoning to today's actual date (so 'due Friday' resolves to a real day rather than being interpreted from phrasing alone) was discussed but not added to the final system prompt.
- Two nodes from an earlier design remain on the canvas, disconnected from the live execution path: an older two-field intake form and its matching capacity-gate IF node. They don't affect behavior since nothing routes through them, but they're worth deleting for a clean submission.
- The Decision Agent's system prompt has a minor copy artifact ("You are a helpful assistantYou are LoopLight...") from an editing pass — cosmetic only, doesn't affect behavior.

11. Learnings and Observations

[Add personal reflections here — what was harder or easier than expected, what you'd do differently, what surprised you about building the agent, how your thinking on 'what counts as agentic' evolved over the build.]

12. What's Explicitly Out of Scope (v2 candidates)

- Todoist (or another external task manager) as a second, human-visible surface for accepted-but-deferred items.
- A Classification Agent that checks new brain-dump items against existing open items to catch duplicates or updates.
- Calendar write-back for event-type items, including free-text deadline parsing into real dates.
- The Flag for Clarification second tool and date-anchoring, both designed above but not yet wired in.
- A weekly ‘focus areas’ mode that recommends a batch for the week rather than a single-session pick.
- Two-way sync so completing a task elsewhere reconciles back into LoopLight's state.

13. Is This Documentation Final?

Functionally, yes — every requirement the handout calls out by name (decides what to do next, calls tools, holds state, hands off to a human, recovers from errors) is documented above with evidence from actual testing, and Section 10 is deliberately honest about the few things that were designed but not shipped, rather than glossing over them. The two things still needed before this is submission-ready: my own reflections in Section 11 (left as a placeholder on purpose, since that's not mine to write for you), and, optionally, the two harmless cleanup items in Section 10 (deleting the orphaned nodes, fixing the prompt typo) — neither blocks grading, but both take under five minutes if you want the workflow file itself to look as clean as the documentation describes it.

LoopLight — Full Flow Diagram

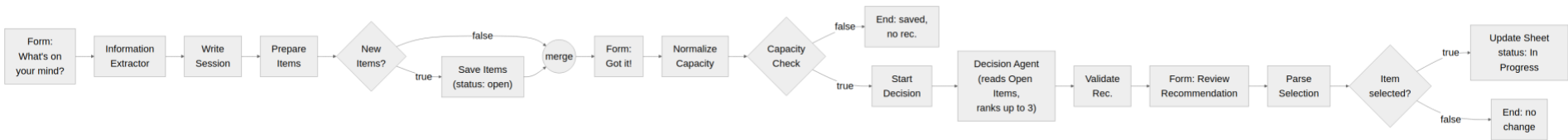


Diagram reflects the actual shipped workflow (LoopLight_Final.json) as of this audit, including the corrected placement of the capacity gate after item-saving rather than before it.